

The Small Short. ¹ You are day trading the GME stock. The price of GME throughout the day is encoded in array $A[1, n]$, i.e., $A[i]$ is the price of GME at time $1 \leq i \leq n$. You can either buy or sell a single stock of GME at each timestep. There is a selling fee $S > 0$ but no buying fee. Find the maximum profit you can achieve, making as many buy/sell transactions as you like.

Solution. Our DP state should encode the current time $i \leq n$ and whether we currently have the stock (represented by a boolean $b \in \{0, 1\}$). Hence, we implement the following function:

$\text{dp}(i, b)$: Given $1 \leq i \leq n$ and $b \in \{0, 1\}$, $\text{dp}(i, b)$ is the maximum profit at time i s.t.:
you currently own the stock if $b = 1$ or you currently don't own the stock if $b = 0$.

$\text{dp}(i, b)$

1. If $i = 1$:

(a) If $b = 0$: output 0. (no stock == 0 profit)

(b) If $b = 1$: output $-A[1]$. (stock == need to purchase the stock)

2. If $b = 0$: output $\max\{\underbrace{\text{dp}(i-1, 1) + A[i] - S}_{\text{sell stock at time } i}, \underbrace{\text{dp}(i-1, 0)}_{\text{sell stock before time } i}\}$

3. If $b = 1$: output $\max\{\underbrace{\text{dp}(i-1, 0) - A[i]}_{\text{buy stock at time } i}, \underbrace{\text{dp}(i-1, 1)}_{\text{buy stock before time } i}\}$

(Cache prior $\text{dp}(j, b')$ calls for any $j \leq i$ and $b' \in \{0, 1\}$).

To output the final result, we run $\text{dp}(n, 0)$. There are $O(n)$ subproblems, each which take $O(1)$ time to compute, so the runtime is $O(n)$.

Justin says: During the PSO, I broke the $\text{dp}(i, b)$ function into two functions, $\text{buy}(i) \cong \text{dp}(i, 1)$ and $\text{sell}(i) \cong \text{dp}(i, 0)$.

¹“Look at them, that’s my quant. - Your what? - My quantitative. My math specialist. Look at them, you notice anything different about them? Look at their face. Look at their grit, I’ll give you a hint, they’re a 381 student. They won a national math competition in Indiana! They don’t even speak Hoosier! Yeah I’m sure of the math.”

I Can't Lie, Justin Has All of Her Songs Downloaded. ² Saylor Twift is getting ready to have another world tour, starting in West Lafayette, Indiana³. She wants to make as much money as possible utilizing a predatory pricing scheme: on some days she will sell low, and on some days, she will sell high.

There are n seats and the concert will happen in T days. On each day $1 \leq t \leq T$, you must choose to set the ticket prices to the *low price* p_L or the *high price* p_H . On any given day,

- If you choose the high price p_H , then the number of purchase requests will be H_t .
- If you choose the low price, then the number of purchase requests will be L_t , where $L_t \geq H_t$ since any buyer who'd buy a high price ticket would also buy a low price ticket.

You are required to fulfill all purchase requests as long as seats remain available. Ticket sales end when all n seats are filled or after $t = T$, i.e., the departure date has been reached. Find the maximum profit you can make.

Solution. Our DP state should encode the remaining seats $m \leq n$ and the time remaining $t \leq T$.

$\text{dp}(m, t)$: Given $0 \leq m \leq n$ and $0 \leq t \leq T$, $\text{dp}(m, t)$ is the maximum profit if there are m available seats and t days remaining for ticket sales.

$\text{dp}(m, t)$

1. If $m = 0$: output 0 (no tickets to sell)
2. If $t = 0$: output 0 (no days of sales remaining).
3. Compute $\text{numL} \leftarrow \min\{m, L_t\}$ and $\text{numH} \leftarrow \min\{m, H_t\}$.
4. Output $\max\{\underbrace{p_H \cdot \text{numH} + \text{dp}(m - \text{numH}, t - 1)}_{\text{sell high}}, \underbrace{p_L \cdot \text{numL} + \text{dp}(m - \text{numL}, t - 1)}_{\text{sell low}}\}$

(Cache prior $\text{dp}(\cdot, \cdot)$ calls).

To output the final result, we run $\text{dp}(n, T)$. There are $O(nT)$ subproblems, each which take $O(1)$ time to compute, so the runtime is $O(nT)$.

²This is a lie - Justin

³This is true; a liar never lies twice but sometimes thrice.